

CADERNO DE APONTAMENTOS

Provisionamento de Infraestrutura AWS com Terraform, EC2 e Docker

Unidade Curricular de DevOps — 3.º Ano, 2.º Semestre

Projeto: devops-projects — Fase 2

Autor: Agnelo

Instituto Superior de Engenharia de Coimbra (ISEC)

Este documento regista, explica e contextualiza, passo a passo, o trabalho prático realizado nesta fase do projeto — desde a autenticação na AWS até à execução de contentores Docker numa instância EC2 provisionada por Terraform.

Índice

- 1. Introdução e objetivos
- 2. Conceitos fundamentais: IaC, Terraform e o papel do AWS CLI
- 3. Verificação do acesso à AWS
- 4. Estrutura do projeto e leitura anotada do código Terraform
- 5. O ciclo de vida do Terraform em prática
- 6. Análise detalhada dos erros encontrados
- 7. O resultado do apply: o que foi efetivamente criado e porquê
- 8. Ligação à instância e configuração do Docker
- 9. Segurança: o que correu mal e como remediar
- 10. Boas práticas e lições retidas
- 11. Próximos passos

1. Introdução e objetivos

Este caderno documenta, de forma organizada e comentada, a segunda fase de um projeto académico de DevOps cujo objetivo é aprender a provisionar infraestrutura em nuvem de forma automatizada e reproduzível. Em vez de criar recursos na AWS clicando manualmente na consola web — abordagem frágil, difícil de repetir e impossível de rever historicamente — o projeto usa Terraform, uma ferramenta de Infrastructure as Code (IaC), para descrever a infraestrutura desejada em ficheiros de texto e deixar que a ferramenta calcule e execute as mudanças necessárias.

O objetivo concreto desta fase foi criar, de forma automatizada: uma instância de máquina virtual (EC2) capaz de correr contentores Docker; um grupo de segurança (Security Group) que define que tráfego de rede é permitido entrar e sair dessa máquina; e um repositório de imagens de contentores (ECR) onde a aplicação, uma vez construída como imagem Docker, poderá ser publicada e mais tarde descarregada pela instância EC2.

Mais do que apresentar o resultado final, este documento tem uma intenção pedagógica: explica o raciocínio por trás de cada comando, interpreta os erros que surgiram durante o processo — que são, tantas vezes, a parte mais instrutiva de qualquer trabalho prático — e regista as decisões de segurança que foram tomadas (e algumas que ainda precisam de correção). O objetivo é que qualquer colega que leia este caderno consiga não apenas repetir os passos, mas compreender genuinamente porque é que cada passo é necessário.

2. Conceitos fundamentais: IaC, Terraform e o papel do AWS CLI

Antes de entrar no código propriamente dito, vale a pena clarificar três conceitos que atravessam todo este trabalho.

2.1. Infrastructure as Code (IaC)

IaC é a prática de descrever a infraestrutura de um sistema — servidores, redes, permissões, bases de dados — através de ficheiros de configuração legíveis por máquina, em vez de a configurar manualmente. Isto traz várias vantagens diretamente visíveis neste projeto: a infraestrutura fica versionada (pode ser guardada em Git, revista, comparada ao longo do tempo), é reproduzível (o mesmo ficheiro cria sempre o mesmo resultado, em qualquer conta ou ambiente), e é auditável (dá para ver exatamente o que mudou e quando).

2.2. Terraform e o conceito de estado

O Terraform é a ferramenta de IaC usada aqui. Funciona por comparação: mantém um registo — o chamado "estado" (state) — do que já foi criado na infraestrutura real, compara-o com o que está descrito nos ficheiros .tf, e calcula o conjunto mínimo de operações (criar, alterar, destruir) necessárias para que a realidade passe a corresponder à descrição. É exatamente esta comparação que se vê no output de um terraform plan, com os símbolos + (criar), ~ (alterar) e - (destruir).

O ficheiro de estado é um dado crítico: se for perdido, o Terraform deixa de saber o que já existe e pode tentar recriar recursos duplicados ou perder o controlo sobre eles. Por isso, este projeto configura logo no início um backend remoto (ficheiro backend.tf, explicado na secção seguinte) que

guarda o estado num bucket S3 em vez de num ficheiro local — decisão importante sobretudo em contexto de equipa.

2.3. O papel do AWS CLI

O AWS CLI (Command Line Interface) é usado aqui não para criar recursos diretamente — essa responsabilidade fica a cargo do Terraform — mas para confirmar que as credenciais estão corretamente configuradas na máquina antes de se avançar. É uma verificação de bom senso: se as credenciais estiverem erradas ou ausentes, é preferível descobri-lo com um comando simples e rápido do que a meio de um terraform apply mais complexo.

3. Verificação do acesso à AWS

O primeiro comando executado foi uma chamada direta ao AWS CLI, sem qualquer argumento:

```
$ aws
```

Isto resulta, como esperado, num erro de validação de parâmetros (ParamValidation), porque a CLI da AWS exige sempre a indicação de um comando (por exemplo sts, ec2 ou s3) seguido de um subcomando. Não é um sinal de mau funcionamento — é simplesmente a forma como a ferramenta foi desenhada, e útil lembrar isto para não perder tempo a diagnosticar um "problema" que não existe.

O comando efetivamente relevante foi este:

```
$ aws sts get-caller-identity
```

O serviço STS (Security Token Service) da AWS devolve a identidade associada às credenciais atualmente configuradas na máquina. A resposta obtida confirmou três coisas essenciais: o identificador único do utilizador IAM (UserId), o número da conta AWS onde os recursos vão ser criados (Account) e o Amazon Resource Name completo do utilizador (Arn), neste caso arn:aws:iam::635096816538:user/devops-user.

Correr este comando antes de qualquer operação com Terraform é uma boa prática que vale a pena manter sempre: garante que se está autenticado, que se está a apontar para a conta certa (é fácil, em ambientes com múltiplos perfis AWS configurados, estar sem querer a atuar sobre a conta errada) e evita que um erro de credenciais apareça de forma confusa muito mais tarde no processo.

4. Estrutura do projeto e leitura anotada do código Terraform

O repositório devops-projects está organizado em pastas por fase do projeto (projeto-devops-fase-1 a projeto-devops-fase-4), o que é uma boa prática em contexto académico: permite mostrar a evolução do trabalho sem misturar configurações de fases diferentes. A pasta de trabalho desta fase, projeto-devops-fase-2, contém quatro ficheiros Terraform: backend.tf, provider.tf, ecr.tf e ec2.tf. Cada um tem uma responsabilidade única e bem definida, o que é também uma boa prática de organização de código — evita ficheiros monolíticos difíceis de navegar.

4.1. backend.tf — onde vive o estado do Terraform

```
terraform {
  backend "s3" {
    bucket  = "terraform-state-agnelo"
    key     = "site/terraform.tfstate"
    region  = "us-east-1"
    encrypt = "true"
  }
}
```

Este bloco não cria nenhum recurso de infraestrutura — configura o próprio funcionamento interno do Terraform. Diz-lhe para guardar o ficheiro de estado (terraform.tfstate) dentro de um bucket S3 chamado terraform-state-agnelo, no caminho site/terraform.tfstate, com o conteúdo encriptado em repouso. A vantagem de um backend remoto como este, em vez do comportamento por omissão do Terraform (guardar um ficheiro terraform.tfstate na pasta local), é dupla: o estado deixa de depender do disco de uma única pessoa, e — em cenários de equipa — o S3 pode ser combinado com mecanismos de bloqueio (locking) que impedem duas pessoas de correr um apply em simultâneo e corromperem o estado.

4.2. provider.tf — que "fornecedor" de infraestrutura estamos a usar

```
provider "aws" {
  region = "us-east-1"
}
```

O bloco provider diz ao Terraform qual a API que vai usar para traduzir os recursos descritos (aws_instance, aws_security_group, etc.) em chamadas reais a um fornecedor de nuvem — neste caso a AWS, na região us-east-1. Este detalhe da região parece trivial, mas foi precisamente a origem de um dos erros mais instrutivos deste projeto (ver secção 6.3): todos os recursos referenciados por ID — como uma VPC — só existem dentro de uma região específica, e se o provider estiver configurado para uma região diferente daquela onde o recurso realmente existe, a AWS recusa-se a encontrá-lo.

4.3. ecr.tf — o repositório de imagens Docker

```
resource "aws_ecr_repository" "ecr_site" {
  name                = "site_prod"
  image_tag_mutability = "MUTABLE"
}
```

O Amazon ECR (Elastic Container Registry) é o equivalente, dentro da AWS, a um Docker Hub privado: um sítio onde se publicam (docker push) e de onde se descarregam (docker pull) imagens de contentores. Este bloco cria um repositório chamado `site_prod`. O atributo `image_tag_mutability = "MUTABLE"` permite que a mesma tag (por exemplo, `latest`) seja sobrescrita por versões mais recentes da imagem — o oposto seria `IMMUTABLE`, mais rigoroso e mais próprio de ambientes de produção maduros, onde se prefere forçar cada build a ter uma tag única e nunca reutilizável, evitando ambiguidade sobre que código está de facto em execução.

4.4. `ec2.tf` — a instância, o grupo de segurança e as suas regras

Este é o ficheiro mais denso do projeto e o que concentra a maior parte dos erros discutidos na secção 6. Divide-se em quatro partes lógicas.

A instância EC2

```
resource "aws_instance" "website_server" {
  ami              = "ami-0354c98ae10b02961"    # Amazon Linux 2 AMI
  instance_type    = "t3.micro"
  key_name          = "chave-site-prod"
  vpc_security_group_ids = [aws_security_group.website_sh.id]
  iam_instance_profile = "ECR-EC2-Role"

  tags = {
    Name           = "website-server"
    ProvidedBy     = "Terraform"
    CreatedBy      = "Agnelo"
    Date           = "2024-09-14"
  }
}
```

O atributo `ami` identifica a imagem de sistema operativo a usar — neste caso uma Amazon Linux 2 AMI, identificada apenas pelo seu ID (as AMIs têm IDs diferentes por região, outro detalhe a ter em conta se este código for reutilizado noutra região). O `instance_type = "t3.micro"` escolhe o tamanho da máquina virtual — `t3.micro` é propositadamente pequeno e está, tipicamente, coberto pelo nível gratuito da AWS, o que faz sentido num contexto de aprendizagem. O `key_name` identifica o par de chaves SSH que vai ser injetado na instância para permitir o acesso remoto seguro (ver secção 8).

O atributo mais interessante do ponto de vista de Terraform é `vpc_security_group_ids = [aws_security_group.website_sh.id]`. Isto não é uma string — é uma referência a outro recurso definido mais abaixo no mesmo ficheiro (`aws_security_group.website_sh`), acedendo ao seu atributo `id`. O Terraform resolve automaticamente a ordem de criação: percebe que a instância depende do grupo de segurança e garante que o grupo é criado primeiro. Este mecanismo — referências entre recursos — é central para perceber tanto o funcionamento correto do Terraform como os erros da secção 6.1.

Por fim, `iam_instance_profile = "ECR-EC2-Role"` associa a esta instância uma função (role) do IAM previamente criada na consola AWS, que lhe concede permissões — por exemplo, para autenticar-se junto do ECR e fazer pull de imagens sem ser necessário embutir credenciais estáticas na máquina. Esta é, de resto, a forma recomendada de dar acesso à AWS a partir de uma instância EC2: usar uma role associada, nunca chaves de acesso fixas guardadas dentro da máquina.

O grupo de segurança (Security Group)

```
resource "aws_security_group" "website_sh" {
  name           = "website-sg"
  description    = "Allow TLS inbound traffic and all outbound traffic"
  vpc_id        = "vpc-0f924f2820e419548"

  tags = { ... }
}
```

Um Security Group funciona como uma firewall com estado (stateful) associada a uma ou mais instâncias. Por si só, este bloco não define nenhuma regra de tráfego — apenas cria o "contentor" onde essas regras vão ser associadas. O atributo `vpc_id` liga o grupo de segurança a uma VPC (Virtual Private Cloud) específica — a rede virtual isolada dentro da qual a instância vai viver. Foi precisamente neste atributo que ocorreram dois dos erros mais relevantes deste projeto, discutidos em detalhe na secção 6.

As regras de entrada (ingress) e saída (egress)

Nas versões mais recentes do provider AWS para Terraform, a forma recomendada de definir regras de tráfego deixou de ser um bloco ingress/egress embutido dentro do próprio `security_group`, passando a usar recursos independentes — `aws_vpc_security_group_ingress_rule` e `aws_vpc_security_group_egress_rule` — cada um associado ao grupo através do atributo `security_group_id`. Esta separação torna o código mais explícito e mais fácil de gerir individualmente (por exemplo, remover uma única regra sem tocar nas restantes).

```
resource "aws_vpc_security_group_ingress_rule" "allow_ssh" {
  security_group_id = aws_security_group.website_sh.id
  cidr_ipv4         = "188.82.120.251/32"
  from_port         = 22
  ip_protocol       = "tcp"
  to_port           = 22
}
```

Esta regra permite tráfego SSH (porta 22) apenas a partir de um único endereço IP — o do computador do autor, identificado com o sufixo `/32`, que em notação CIDR significa "exatamente este endereço e nenhum outro". Isto é uma aplicação direta do princípio do menor privilégio: em vez de abrir a porta de administração remota ao mundo inteiro (`0.0.0.0/0`), restringe-se o acesso ao mínimo estritamente necessário. As duas regras seguintes, para as portas 80 (HTTP) e 443 (HTTPS), usam propositadamente `0.0.0.0/0`, porque um servidor web deve, por definição, estar acessível a qualquer visitante.

Por fim, a regra de saída (egress) permite todo o tráfego de saída para qualquer destino, usando `ip_protocol = "-1"` — o valor especial que, nas APIs da AWS, significa "todos os protocolos". É uma configuração comum e razoável: normalmente restringe-se o que pode entrar numa máquina, mas é menos crítico restringir o que ela pode iniciar para fora, a não ser em ambientes com requisitos de segurança muito elevados (por exemplo, para prevenir exfiltração de dados).

5. O ciclo de vida do Terraform em prática

O fluxo de trabalho seguido segue, com pequenas variações, o ciclo de vida padrão de qualquer projeto Terraform. Vale a pena explicar o papel de cada comando individualmente, porque cada um resolve um problema diferente e usá-los na ordem errada é uma fonte comum de confusão para quem está a aprender.

5.1. terraform fmt

Reformata automaticamente os ficheiros .tf segundo o estilo canónico da linguagem HCL — alinha os sinais de igual, normaliza a indentação. Não altera o significado do código, apenas a sua apresentação. É seguro correr sempre, e foi corrido várias vezes ao longo da sessão — um sinal de disciplina saudável, sobretudo em código partilhado, onde um estilo consistente facilita a leitura por outras pessoas.

5.2. terraform init

Prepara a pasta de trabalho: descarrega os plugins do provider necessário (neste caso hashicorp/aws, versão 6.64.0, tal como registado no ficheiro de bloqueio .terraform.lock.hcl) e configura a ligação ao backend remoto definido em backend.tf. É preciso voltar a correr este comando sempre que se altera a configuração do backend ou se adicionam novos providers — o próprio Terraform avisa disso na sua mensagem de sucesso.

5.3. terraform validate

Verifica a sintaxe e a coerência interna do código — por exemplo, se uma referência aponta para um recurso que existe, ou se um bloco está bem formado — sem nunca contactar a AWS. É por isso uma verificação rápida e sem custo, útil como primeiro diagnóstico quando algo corre mal, tal como se vê nas secções 6.1 e 6.2, onde terraform validate foi corrido logo a seguir a um erro para confirmar que o problema (nesses casos) já não era de sintaxe.

5.4. terraform plan

Este é o comando que efetivamente contacta a AWS (para consultar o estado atual dos recursos) e calcula a diferença entre esse estado e o que está descrito no código. O resultado é um plano de execução, anotado com os símbolos + (recurso a criar), ~ (a alterar) e - (a destruir). É fundamental correr sempre um plan antes de um apply — sobretudo num ambiente de produção — para se poder rever, com calma, exatamente que mudanças vão ser feitas antes de as confirmar.

5.5. terraform apply

Executa de facto as mudanças calculadas pelo plan, pedindo sempre uma confirmação explícita (escrever "yes") antes de avançar — uma proteção deliberada contra execuções acidentais. É possível saltar esta confirmação com a flag -auto-approve, mas isso não foi usado neste projeto, e bem: em fase de aprendizagem, ler o plano antes de confirmar é como se aprende a interpretar os efeitos secundários de cada alteração ao código.

5.6. terraform destroy -target=<recurso>

Foi usado uma vez neste projeto, para remover isoladamente o repositório ECR (terraform destroy -target=aws_ecr_repository.ecr_site), sem afetar a instância EC2 ou o grupo de segurança já criados. A flag -target é poderosa mas o próprio Terraform avisa, no seu output, que não deve ser usada por rotina — apenas em situações excepcionais de recuperação de erros — porque um plano feito com -target deixa de refletir o estado completo da configuração, podendo mascarar outras mudanças pendentes. Vale a pena registrar este aviso como um alerta a levar a sério em projetos maiores.

6. Análise detalhada dos erros encontrados

Nenhum destes erros foi, no fundo, um "bug" — todos resultam de más interpretações de como a linguagem HCL e a AWS funcionam. É exatamente por isso que vale a pena dissecá-los com calma: entender a causa raiz de cada um evita repeti-los no futuro, em vez de simplesmente decorar a correção pontual que foi aplicada.

6.1. "Invalid reference" — valores em falta com aspas

Os dois primeiros erros encontrados tiveram exatamente a mesma causa. O primeiro surgiu na linha:

```
vpc_id = vpc-0f924f2820e419548
```

E o segundo, momentos depois, numa linha muito semelhante:

```
ami = ami-0354c98ae10b02961
```

A mensagem de erro do Terraform foi clara, ainda que exija alguma familiaridade com HCL para ser totalmente compreendida: "A reference to a resource type must be followed by at least one attribute access, specifying the resource name." Isto acontece porque, em HCL, um valor escrito sem aspas não é tratado como texto literal — é interpretado como uma expressão, e mais especificamente como uma referência a outro bloco de recursos, no formato `tipo_de_recurso.nome_do_recurso.atributo` (por exemplo, `aws_security_group.website_sh.id`, como se viu na secção 4.4).

Ao escrever `vpc_id = vpc-0f924f2820e419548`, o Terraform tentou interpretar isto como uma referência a um recurso do tipo "vpc" — o que não faz sentido nem sintaticamente (falta o segundo ponto e o nome do atributo), daí o erro. A correção é simples mas o raciocínio por trás dela é o que importa reter: sempre que se quer que o Terraform trate um valor como uma cadeia de texto literal — um ID copiado da consola AWS, um nome, um caminho — esse valor tem de estar entre aspas duplas. Só quando não há aspas é que o Terraform o interpreta como uma expressão dinâmica (uma referência, uma função, uma variável).

A correção aplicada foi trivial: `vpc_id = "vpc-0f924f2820e419548"` e `ami = "ami-0354c98ae10b02961"`. Esta é provavelmente a regra mais importante de reter deste caderno inteiro para quem está a começar com Terraform: strings literais sempre entre aspas; referências a outros recursos, nunca entre aspas.

6.2. Region vs. VPC ID — recursos existem apenas dentro da sua região

Com o erro de sintaxe corrigido, o terraform apply avançou até criar o repositório ECR com sucesso, mas falhou ao tentar criar o grupo de segurança, com este erro devolvido diretamente pela API da AWS (já não pelo próprio Terraform):

```
Error: creating Security Group (website-sg): operation error EC2:
CreateSecurityGroup,
api error InvalidVpcID.NotFound: The vpc ID 'vpc-0f924f2820e419548' does
not exist
```

Este é um erro de um tipo diferente do anterior: já não é um problema de sintaxe (o terraform validate, corrido logo a seguir, confirmou que a configuração era sintaticamente válida), mas um problema

semântico — o valor está bem escrito, mas não corresponde a nada que exista de facto do lado da AWS.

A causa mais provável, olhando para o output completo da sessão, está relacionada com a região. Nas primeiras tentativas de `plan` e `apply`, os recursos mostravam `region = "us-east-2"`, enquanto que o ficheiro `provider.tf` tinha sido entretanto ajustado para `us-east-1`. Este detalhe é fundamental de compreender: a AWS não tem VPCs globais — cada VPC pertence a uma região específica, e um ID de VPC copiado da consola enquanto se navegava, por exemplo, na região Ohio (`us-east-2`) simplesmente não existe do ponto de vista de um pedido feito à região Norte da Virgínia (`us-east-1`), mesmo que o texto do ID pareça correto.

A resolução, tal como registada na sessão, passou por garantir consistência entre três pontos que têm de estar sempre alinhados: a região definida no `provider.tf`, a região efetivamente usada para consultar/copiar o ID da VPC na consola AWS, e a região onde, de facto, a instância e o grupo de segurança vão ser criados. Depois desse alinhamento (com tudo a apontar para `us-east-1`), o `apply` seguinte criou os seis recursos com sucesso.

Fica como lição mais geral: sempre que um ID copiado da consola AWS "não existe" apesar de parecer correto, a primeira coisa a verificar não é o ID em si, mas a região onde o pedido está a ser feito.

6.3. Timeout de rede ao validar credenciais

```
Error: validating provider credentials: retrieving caller identity from
STS: ...
dial tcp: lookup sts.us-east-1.amazonaws.com ...: i/o timeout
```

Este erro não tem origem no código Terraform nem na configuração da AWS — é uma falha transitória de resolução de DNS ou de conectividade de rede, algo relativamente comum em ambientes WSL (Windows Subsystem for Linux), sobretudo quando há mudanças de rede (Wi-Fi, VPN, troca de interface de rede) entre comandos. A confirmação de que era um problema transitório veio da forma mais simples possível: repetir exatamente o mesmo comando resolveu o problema na tentativa seguinte, sem qualquer alteração de configuração.

Vale a pena reter este tipo de erro como uma categoria à parte: antes de começar a alterar código ou configuração à procura de uma causa, é sensato verificar primeiro se o erro é de facto reprodutível ou se foi apenas uma falha pontual de rede.

6.4. Chave SSH com permissões demasiado abertas

Ao tentar ligar-se pela primeira vez à instância recém-criada, obteve-se este aviso, seguido de falha de autenticação:

```
WARNING: UNPROTECTED PRIVATE KEY FILE!
Permissions 0777 for 'chave-site-prod.pem' are too open.
Load key "chave-site-prod.pem": bad permissions
ec2-user@...: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

O cliente SSH recusa-se deliberadamente a usar uma chave privada cujas permissões de ficheiro permitam leitura por outros utilizadores do sistema — no caso, 0777 significa que qualquer utilizador da máquina local poderia ler o ficheiro. Esta é uma proteção do próprio protocolo SSH, e não uma particularidade da AWS: uma chave privada só deve poder ser lida pelo seu dono.

A correção foi copiar a chave para a pasta convencional `~/.ssh/` e restringir as permissões com `chmod 600`, que concede leitura e escrita apenas ao utilizador dono do ficheiro, e nenhum acesso a mais ninguém:

```
$ cp chave-site-prod.pem ~/.ssh/chave-site-prod.pem
$ chmod 600 ~/.ssh/chave-site-prod.pem
```

Depois desta correção, a ligação SSH foi estabelecida com sucesso. É uma boa prática a interiorizar de forma permanente: sempre que se recebe ou copia uma chave privada para uma nova máquina, o primeiro comando a correr deve ser sempre `chmod 600`, antes mesmo de tentar usá-la.

6.5. Permissão negada ao usar o Docker sem sudo

```
$ docker ps
permission denied while trying to connect to the Docker daemon socket
at unix:///var/run/docker.sock ...
```

O daemon do Docker corre com privilégios de root e comunica com os clientes através de um socket Unix local (`/var/run/docker.sock`) cujo acesso é, por omissão, restrito ao próprio root e aos membros do grupo `docker`. O utilizador `ec2-user`, imediatamente após a instalação do pacote, ainda não pertence a esse grupo — daí a permissão negada. A solução imediata usada na sessão foi anteceder o comando com `sudo`, que funciona porque o root tem sempre acesso ao socket:

```
$ sudo docker ps
```

Isto resolve o problema pontualmente, mas não é a solução mais prática a prazo — obrigaria a escrever `sudo` antes de cada comando Docker. A solução permanente, ainda por aplicar neste projeto (ver secção 11), é adicionar o utilizador ao grupo `docker` com `sudo usermod -aG docker ec2-user`, terminando e reabrindo a sessão SSH para que a nova pertença ao grupo seja reconhecida. Vale a pena, no entanto, sublinhar uma nota de segurança sobre esta prática, desenvolvida com mais detalhe na secção 9: pertencer ao grupo `docker` é, na prática, equivalente a ter privilégios de root nessa máquina, porque o daemon Docker corre como root e um utilizador com acesso ao seu socket pode, por exemplo, montar o sistema de ficheiros raiz dentro de um contentor. Não é uma permissão a atribuir de forma leviana em máquinas partilhadas ou multiutilizador.

7. O resultado do apply: o que foi efetivamente criado e porquê

Depois de resolvidos os erros da secção anterior, o terraform apply criou com sucesso seis recursos. Vale a pena percorrê-los não apenas como uma lista de identificadores, mas com uma explicação de porque é que cada um existe e como se relaciona com os restantes.

O grupo de segurança website-sh (identificador sg-045f9e4ea71124228) foi criado primeiro, precisamente porque a instância EC2 depende dele através da referência vpc_security_group_ids explicada na secção 4.4 — o Terraform respeitou automaticamente esta dependência sem que fosse necessário indicá-la explicitamente com um bloco depends_on.

As quatro regras de tráfego foram criadas de seguida: a regra allow_ssh, que abre apenas a porta 22 e apenas para o endereço IP do autor; as regras allow_http e allow_https, que abrem as portas 80 e 443 a qualquer origem, como é próprio de um servidor web público; e a regra allow_all_outbound, que permite à instância iniciar ligações para qualquer destino. Em conjunto, estas quatro regras definem exatamente a superfície de exposição da máquina: administração restrita a uma única origem confiável, serviço web aberto ao público, e liberdade total de saída.

Por fim, a instância EC2 website_server foi criada (identificador i-09442f5c6b13f25d0), com o IP público 100.53.195.3 atribuído automaticamente pela AWS — este IP muda se a instância for parada e reiniciada, a não ser que se associe um Elastic IP, o que não foi feito nesta fase e é uma limitação a ter em conta caso se pretenda um endereço estável a prazo.

O repositório ECR site_prod, criado numa fase anterior da sessão, foi depois deliberadamente destruído com terraform destroy -target=aws_ecr_repository.ecr_site — um teste intencional do comando de destruição seletiva — e recriado no final através de um novo apply, desta vez sem qualquer erro, já com a configuração de região estabilizada.

8. Ligação à instância e configuração do Docker

Com a infraestrutura criada, o passo seguinte foi ligar-se à instância por SSH e preparar o ambiente de execução de contentores.

```
$ ssh -i ~/.ssh/chave-site-prod.pem ec2-user@100.53.195.3
```

A instância corre Amazon Linux 2023, uma distribuição da AWS baseada em RPM (usa o gestor de pacotes dnf/yum, e não apt, próprio de distribuições baseadas em Debian/Ubuntu — um detalhe que gerou uma tentativa falhada de sudo apt update no início da sessão, corrigida de imediato para sudo yum update).

A instalação do Docker seguiu três passos bem definidos, cada um com um propósito distinto:

```
$ sudo yum install docker      # instala o pacote e as suas dependências
$ sudo systemctl enable docker # regista o serviço para arrancar
                                automaticamente no boot
$ sudo systemctl start docker  # arranca o serviço imediatamente, nesta
                                sessão
```

É importante distinguir `enable` de `start`: `enable` garante que o Docker vai arrancar sozinho da próxima vez que a instância reiniciar (por exemplo, após uma paragem/arranque na consola AWS), enquanto `start` arranca o serviço imediatamente, sem esperar por um reboot. Fazer só um dos dois é um erro comum — só `enable` deixa o Docker parado até ao próximo reboot; só `start` faz o Docker esquecer-se de arrancar sozinho na próxima vez.

O teste final, `docker ps`, revelou o erro de permissões já analisado em detalhe na secção 6.5, contornado nesta sessão com `sudo docker ps`, que devolveu uma lista vazia de contentores em execução — resultado esperado, já que nenhum contentor tinha ainda sido lançado. A instância está, a partir deste ponto, pronta para receber a imagem Docker da aplicação, que deverá ser publicada no repositório ECR criado anteriormente e depois descarregada (`docker pull`) e executada (`docker run`) nesta máquina — trabalho que corresponde às fases seguintes do projeto.

9. Segurança: o que correu mal e como remediar

Esta secção merece destaque próprio porque, ao contrário dos erros da secção 6 — que impediram o trabalho de avançar e por isso foram corrigidos de imediato — os dois problemas seguintes não bloquearam nada tecnicamente, o que os torna mais fáceis de ignorar e, por isso, mais importantes de assinalar explicitamente.

Exposição da chave privada SSH

A determinada altura da sessão, o comando `cat *` foi usado para inspecionar rapidamente o conteúdo de todos os ficheiros da pasta de trabalho. Isto incluiu, sem intenção, o ficheiro `chave-site-prod.pem` — a chave privada RSA usada para autenticação SSH na instância — cujo conteúdo completo foi impresso em texto simples no terminal. Uma chave privada nunca deve ser impressa, partilhada ou colada em lado nenhum: quem a possuir consegue autenticar-se na instância como se fosse o dono legítimo, sem precisar de mais nada. Se este terminal alguma vez for copiado para um sítio acessível a terceiros — um repositório, um fórum, uma conversa de chat — a chave deve ser considerada comprometida a partir desse momento, independentemente de a instância ainda estar ou não acessível através dela.

A recomendação prática, e prioritária, é rodar esta chave: eliminar o par de chaves `chave-site-prod` na consola AWS (secção EC2 → Key Pairs), gerar um novo par, e voltar a associá-lo à instância — o que, tratando-se de EC2, normalmente implica recriar a instância com a nova chave, já que a AWS não permite trocar o par de chaves de uma instância já em execução sem recorrer a passos adicionais (por exemplo, substituir manualmente a chave pública autorizada dentro da máquina, via um acesso já existente). Vale também a pena adicionar uma entrada `*.pem` ao ficheiro `.gitignore` do repositório, como proteção estrutural contra o erro, mais grave ainda, de um dia se fazer commit acidental desta chave para um repositório Git.

Permissões do ficheiro de chave demasiado abertas

Antes de ser corrigido (secção 6.4), o ficheiro `chave-site-prod.pem` tinha permissões `0777` — leitura, escrita e execução para todos os utilizadores da máquina local. Embora o próprio cliente SSH tenha recusado usar a chave nesse estado (o que, na prática, funcionou como uma rede de segurança), o princípio geral mantém-se: qualquer ficheiro de chave privada deve, assim que criado ou copiado, receber de imediato permissões `600`, sem depender de o SSH vir a recusá-lo mais tarde.

Por fim, uma nota preventiva relacionada com o ponto discutido na secção 6.5: caso se avance com a adição do utilizador `ec2-user` ao grupo `docker` para deixar de ser necessário o `sudo` em cada comando, é importante registar que essa alteração concede, na prática, privilégios equivalentes a `root` nessa instância. Não é uma preocupação que invalide a prática — é comum e aceitável em instâncias de desenvolvimento ou de uso pessoal — mas é uma decisão consciente a tomar, não um detalhe a ignorar, sobretudo se esta máquina alguma vez vier a ser partilhada por mais do que uma pessoa.

10. Boas práticas e lições retidas

Reunindo o que foi discutido ao longo deste caderno, ficam registadas as práticas que mais valeu a pena reter desta sessão de trabalho, e que fazem sentido levar para as fases seguintes do projeto.

Correr `terraform fmt` e `terraform validate` antes de cada `plan` tornou-se, ao longo da sessão, um hábito natural — e um hábito que compensa, porque apanha erros de sintaxe e de coerência interna sem qualquer custo de tempo ou de chamadas à AWS. É sempre preferível apanhar um erro no `validate`, em segundos, do que a meio de um `apply`, já com recursos parcialmente criados.

A distinção entre valores literais (sempre entre aspas) e referências a outros recursos (nunca entre aspas) revelou-se a fonte de erro mais comum nesta sessão, e é provavelmente a regra de HCL mais importante para quem está a começar. Da mesma forma, ficou claro que qualquer ID copiado da consola AWS — de uma VPC, de uma subnet, de uma imagem AMI — só é válido dentro da região onde foi originalmente consultado, pelo que confirmar a região é sempre o primeiro passo de diagnóstico quando a AWS reporta que um recurso "não existe".

Do lado da segurança, as duas práticas a manter de forma disciplinada daqui em diante são: nunca imprimir o conteúdo de um ficheiro de chave privada no terminal (usar `chmod` diretamente, sem necessidade de o inspecionar visualmente), e manter `*.pem` e `*.tfstate` fora do controlo de versões, através de um `.gitignore` bem configurado desde o início do projeto — não apenas quando um problema já aconteceu.

Por fim, o uso pontual de `-target` no `destroy` demonstrou uma funcionalidade útil para situações excecionais, mas também deixou claro, pelo próprio aviso do Terraform, que não deve tornar-se hábito: o fluxo normal, sem `-target`, é o que garante que o plano apresentado reflete sempre a totalidade da configuração.

11. Próximos passos

Com a infraestrutura base criada e validada, ficam identificadas as ações que naturalmente dão continuidade a este trabalho nas fases seguintes do projeto.

- Rodar a chave `chave-site-prod`, considerando-a comprometida após a exposição registada na secção 9, e atualizar a instância (ou recriá-la) com o novo par de chaves.
- Adicionar `*.pem` e `*.tfstate` ao `.gitignore` do repositório, caso ainda não esteja feito.
- Adicionar `ec2-user` ao grupo `docker` de forma permanente (`sudo usermod -aG docker ec2-user`), com consciência da implicação de segurança discutida na secção 9.
- Construir a imagem Docker da aplicação e publicá-la no repositório ECR `site_prod` (`docker build`, `docker tag`, `docker push`).
- Configurar o `pull` e o `run` da imagem na instância EC2 — manualmente para validar o fluxo, e mais tarde de forma automatizada, por exemplo através de `user_data` no próprio Terraform ou de um pipeline de CI/CD.
- Avaliar a associação de um Elastic IP à instância, caso se pretenda um endereço público estável entre reinícios.

- Documentar as Fases 3 e 4 do projeto seguindo a mesma estrutura de análise usada neste caderno.

— Fim do caderno de apontamentos da Fase 2 —